PS/2 Mouse

Also be sure to read Mouse Input.

Overview

The PS/2 Mouse is a device that talks to a PS/2 controller using serial communication. Ideally, each different type of PS/2 controller driver should provide some sort of standard/simple "send byte/receive byte" interface, and the PS/2 Mouse driver would use this interface without caring about lower level details (like what type of PS/2 controller the device is plugged into).

Mouse Device Over PS/2

Here is the table of command a generic PS/2 compatible mouse understands:

Standard PS/2 Mouse Commands		
Byte	**Data byte**	**Description**
0xE6	None	Set Scaling 1:1
0xE7	None	Set Scaling 2:1
0xE8	**Byte** / **Resolution** 00 / 1 count/mm 01 / 2 count/mm 02 / 4 count/mm 03 / 8 count/mm	Set Resolution
0xE9	None	Status Request
0xEA	None	Set Stream Mode
0xEB	None	Read Data
0xEC	None	Reset Wrap Mode
0xEE	None	Set Wrap Mode
0xF0	None	Set Remote Mode
0xF2	None	Get Device ID. See Detecting PS/2 Device Types for the response bytes.
0xF3	Sample rate	Set Sample Rate, valid values are 10, 20, 40, 60, 80, 100, and 200.
0xF4	None	Enable Data Reporting
0xF5	None	Disable Data Reporting
0xF6	None	Set Defaults
0xFE	None	Resend
0xFF	None	Reset (**Note: After the result of the power-on test is sent, the mouse sends its ID (0x00)**)

The most common command reply is *0xFA* from the master (mouse), which means acknowledge. You may then get a variable number of bytes afterwards depending on the command. You may also receive other command replies which may state that the master (mouse) has encountered an error decoding your command. For a more detailed list check out some of the links above or look through the Linux source tree.

First, you have to enable the mouse on the PS/2 bus. This requires sending one byte which is clocked over the PS/2 interface. You will then get a response regarding the result. By sending *0xF4* (Enable Data Reporting) the mouse should reply back with a *0xFA* which means acknowledgement. Then afterwards as the mouse pointer is moved it will send back the generic packet format like below. Unless you enable an enhanced mode for the mouse (non-standard) this is what you will get when ever the mouse is moved.

Generic PS/2 Mouse Packet Bits									
BYTE	**7**	**6**	**5**	**4**	**3**	**2**	**1**	**0**
0	yo	xo	ys	xs	1	bm	br	bl
1	X-Axis Movement Value							
2	Y-Axis Movement Value							

Code	Description
yo	Y-Axis Overflow
xo	X-Axis Overflow
ys	Y-Axis Sign Bit (9-Bit Y-Axis Relative Offset)
xs	X-Axis Sign Bit (9-Bit X-Axis Relative Offset)
1	Always One
bm	Button Middle (Normally Off = 0)
br	Button Right (Normally Off = 0)
bl	Button Left (Normally Off = 0)

Each X and Y axis value is relative. The mouse device does not track its location in absolute coordinates. This should also be apparent by the 9-bit values. Instead, it sends back saying I moved this far to the left, to the right, down, or up. To keep track of a mouse position you need to accumulate these relative offsets into a absolute position offset in your code:

```
mouse_x = mouse_x + mouse_packet_rel_x
mouse_y = mouse_y + mouse_packet_rel_y
```

Being these 9-bit values are signed the above pseudo would work.

Also, if you simply read the X- or Y-Axis Movement Value fields you will get an 8-bit unsigned value. Which, if used as unsigned will yield incorrect behavior. If you convert it into a signed 8-bit value you will get behavior that is similar to correct, but strange artifacts will appear when the mouse is moved fast. The correct way to produce a 9-bit or greater signed value is as follows:

```
state = first_byte
d = second_byte
rel_x = d - ((state << 4) & 0x100)
d = third_byte
rel_y = d - ((state << 3) & 0x100)
```

The pseudo code above will cause *((state << 4) & 0x100)* to equal *0x100* only if the signed bit (9'th bit stored in the first byte) is set. If the 9'th bit is set then the value is deemed negative, but the value in *second_byte* is not stored in one or two's complement form. It is instead stored as a positive 8-bit value. So, if *second_byte* is say a *2* then it will become *2 minus 0* since the negative (9'th bit) is off. But, if it is on then it will become *2 minus 0x100* which will produce the twos complement, or *-2*. It will also cause the register to be correctly sign extended no matter its size.

Set Sample Rate Example

To set the sample rate for example, which is a command with a data byte, one would need to do:

```
outb(0xD4, 0x64);                       // tell the controller to address the mouse
outb(0xF3, 0x60);                       // write the mouse command code to the controller's data port
while(!(inb(0x64) & 1) asm("pause"); // wait until we can read
ack = inb(0x60);                        // read back acknowledge. This should be 0xFA
outb(0xD4, 0x64);                       // tell the controller to address the mouse
outb(100, 0x60);                        // write the parameter to the controller's data port
while(!(inb(0x64) & 1) asm("pause"); // wait until we can read
ack = inb(0x60);                        // read back acknowledge. This should be 0xFA
```

Mouse Extensions

Here, an example of mouse that supports extensions. To maintain backwards compatibility you should have to activate these features through the PS/2 bus. Various mouse devices use different ways. Linux mouse drivers for example sometimes handle multiple different devices which all share the same standard packet format above, or at least support the compatibility mode described above.

Z-axis

To enable the Intellimouse Z-axis extension, you have to set some magic into the sample rate:

```
set_mouse_rate(200);  // see the example above
set_mouse_rate(100);
set_mouse_rate(80);
mouseid = identify(); // see Get Device ID, 0xF2
```

After that the mouse should not return Mouse ID *0*, but *3*, and will send 4 bytes data packages as follows:

Intellimouse #1 PS/2 Mouse Packet Bits									
BYTE	**7**	**6**	**5**	**4**	**3**	**2**	**1**	**0**
0	yo	xo	ys	xs	1	bm	br	bl
1	X-Axis Movement Value							
2	Y-Axis Movement Value							
3	Z-Axis Movement Value							

The Z-axis Movement Value is in "2's complement" format. Valid values are -8 to +7. Other bytes are identical to the PS/2 packet.

5 buttons

To enable the 4th and 5th buttons, first you have to try to enable Z-axis, and you can only follow with this if the identification returned *3*.

```
if(mouseid == 3) {
  set_mouse_rate(200);
  set_mouse_rate(200);
  set_mouse_rate(80);
  mouseid = identify();
}
```

If this was successful, the identify command should now return Mouse ID *4*, and the 4 bytes packets will look like this:

Intellimouse #2 PS/2 Mouse Packet Bits								
BYTE	**7**	**6**	**5**	**4**	**3**	**2**	**1**	**0**
0	yo	xo	ys	xs	1	bm	br	bl
1	X-Axis Movement Value							
2	Y-Axis Movement Value							
3	0	0	b5	b4	Z-Axis Movement Value			

Here the X-Axis Movement Value is stored only on the low 4 bits and the last byte is not sign extended. Bits 4 and 5 represents the pressed status of buttons 4 and 5 in that order, same as with bm, br and bl.

Emulation

Normally bochs does only emulate generic PS/2 mouse. To make bochs to handle Z-axis, you should set the proper mouse type in your bochrc file:

```
mouse: type=imps2, enabled=1
```

If you did the sample rate magic right, then you should see this on the bochs console as soon as you issue the last set sample rate command:

```
0000000000i[KBD   ] wheel mouse mode enabled
```

Qemu on the other hand understands not only the Z-axis mode, but the 5 buttons mode too, and returns Mouse ID 4 if you did everything right.

See Also

Articles

- PS/2
- "8042" PS/2 Controller
- PS/2 Keyboard

External Links

- www.Computer-Engineering.org/ps2mouse
- users.utcluj.ro/~baruch/sie/labor/PS2/PS-2_Mouse_Interface.htm

Implementations

- Linux (C,GPL)

Categories: Human Interface Device | Common Devices | Hardware Interfaces